

Model Training Data Guide — All 5 Models

SIH 2026 · PS 26009 · MOIL Digital Mine Portal — Dongri Buzurg Prototype

A model learns a pattern by looking at many examples of input to correct-answer pairs. You feed it these paired examples (training data), it learns the pattern, and it can then predict answers for new inputs it hasn't seen. Every supervised model needs two things: **features** (the input information) and **labels/targets** (the correct answer being taught). Without labels, a supervised model cannot be trained — that distinction matters throughout this guide.

Model 1 — Prospectivity Model (Random Forest)

What it learns: given a grid cell's features, is manganese likely present?

Features needed (inputs)

- NDVI value (or anomaly) for that cell
- Soil moisture value
- Land surface temperature (LST) value
- Whether the cell falls inside a gondite/manganese-bearing rock formation (yes/no, from Geology 50k layer)
- Slope / elevation (from DEM)

Labels needed

Whether the cell falls inside the known Dongri Buzurg lease boundary (1) or outside it (0) — sourced from the NGDR lease polygon already pulled.

How to build the dataset

- 1 Overlay a grid (e.g. 100m x 100m) over the bounding box around the mine.
- 2 For each cell, extract NDVI / soil-moisture / LST / slope values (rasterio or geopandas can sample a raster at a location).
- 3 For each cell, check if it falls inside the lease polygon (geopandas spatial join / point-in-polygon) — label 1 or 0.
- 4 Result: one row per cell, columns = [NDVI, soil_moisture, LST, in_gondite_formation, slope, label].
- 5 Train scikit-learn's RandomForestClassifier on this table.
- 6 Hold back ~20% of labeled cells to test whether the model learned a real pattern, not memorization.

Extra data still needed

Nothing exotic — lease boundary and geology layer are already sourced from NGDR. Main remaining task: pull the satellite raster values (NDVI, soil moisture, LST) for the grid.

Minimal code sketch

```
import geopandas as gpd, rasterio
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

grid_cells['label'] = grid_cells.within(lease_polygon.union_all()).astype(int)
X = grid_cells[['ndvi', 'soil_moisture', 'lst', 'in_gondite', 'slope']]
y = grid_cells['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = RandomForestClassifier(n_estimators=200)
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

Model 2 — Production Forecasting (XGBoost)

What it learns: given a day/month's conditions, what will production be?

Features needed

- Equipment availability/downtime (numeric, % uptime)
- Blasting delay (numeric, days delayed)
- Rainfall (numeric, mm)
- NDVI (numeric)
- Soil moisture (numeric)
- Month-of-year / seasonality flag
- Past production (a lag feature — yesterday's/last month's production)

Labels needed

Actual production output for that same day/month (tons of ore).

The catch

There is no public real historical daily production + equipment log for Dongri Buzurg. This dataset must be synthesized — a plausible history following realistic patterns, not pure randomness.

How to build a synthetic dataset (beginner-friendly)

- 1 Decide a time range, e.g. 3 years of daily records (~1,000 rows).
- 2 Generate each row with a formula, not pure randomness (see code sketch below).

- 3 Generate equipment_uptime, rainfall, blasting_delay within realistic ranges (e.g. uptime 70-100%; rainfall higher June-Sept to mimic monsoon).
- 4 This bakes in real cause-effect relationships — honestly describable as "simulated but causally realistic."
- 5 Train XGBRegressor (from the xgboost library) on this table to predict production from the other columns.

Extra data to find (real, not synthetic)

Actual rainfall data for Bhandara district (IMD) — this part can be real even while equipment/downtime stays synthetic, giving the dataset a partially grounded, defensible basis.

Minimal code sketch

```
base_production = 50 # tons/day baseline for a small opencast mine
production = (base_production
* (equipment_uptime / 100) # downtime reduces output
* (1 - rainfall_impact) # heavy rain reduces output
* (1 - blasting_delay_impact) # delays reduce output
+ random_noise) # small random variation

from xgboost import XGBRegressor
model = XGBRegressor()
model.fit(X_train, y_train)
```

Model 3 — Shortfall Prediction

No training needed. Compare Model 2's predicted output to the planned target and classify:

- Gap < 5% → Low risk
- Gap 5-15% → Medium risk
- Gap > 15% → High risk

No data collection needed for this piece — it is arithmetic on Model 2's output.

Model 4 — Cause Analysis (SHAP)

No new training needed. SHAP explains an already-trained model (Model 2's XGBoost). It identifies which feature (rainfall? downtime? blasting?) contributed most to a specific prediction. No extra data needed beyond Model 2 already existing.

```
import shap
explainer = shap.TreeExplainer(trained_xgboost_model)
shap_values = explainer.shap_values(input_row)
```

Model 5 — Corrective Action Recommendation

Mostly a rule table written by hand, not something trained on data:

```
if top_cause == 'equipment_downtime':  
    recommend('Reallocate equipment from Site X')  
if top_cause == 'rainfall':  
    recommend('Advance blasting schedule')
```

A more "ML" version could train a small decision tree on synthetic "action taken -> did the gap close" data, but for a 6-day build a clean rule table is faster and just as demo-effective. No extra data hunting required.

Summary — Data Still Needed

Data	Real or Synthetic	Source
NDVI, soil moisture, LST per grid cell	Real	Bhuvan / MODIS / Bhoonidhi
Lease boundary + geology labels	Real	NGDR (already pulled)
Slope / elevation	Real	SRTM DEM (USGS EarthExplorer, free)
Rainfall history for Bhandara	Real	IMD
Equipment downtime, blasting delay, daily production history	Synthetic	Generated via Python script, calibrated to MOIL Annual Report figures

Priority next step: have one teammate write the synthetic-data-generation script first, since Models 2, 3, 4, and 5 all depend on it existing — it is the single biggest unblocker for the ML/backend track.